



Proyecto 2:

St4ck Over-Pig





Objetivos

General

- ☐ Traducir un lenguaje de alto nivel a un lenguaje de bajo nivel.

Específicos

- ☐ Validar el lenguaje de alto nivel, tanto de manera léxica, sintáctica y semántica antes de iniciar el proceso de compilación.
- ☐ Generar las estructuras adecuadas para tener una representación intermedia para el proceso de compilación.
- ☐ Poner en práctica los conceptos de stack y heap en el proceso de compilación.
- ☐ Utilizar el código intermedio para obtener una traducción exitosa a un lenguaje de bajo nivel.



Descripción de la actividad

... Zzz... ZZZZZZzzzzzz...

Mientras los cerdos dormían, la resistencia se preparaba para volver a intentar establecer comunicación con los zetarianos...

3...2...1... (si, una cuenta regresiva de nuevo)

Un halo de luz se proyectó en la cañonera del salón 1, esta vez, el mensaje había sido transmitido en código intermedio correctamente, la respuesta de los zetarianos no tardó en llegar. El mensaje decía lo siguiente:

'Sus domestica' son una raza invasora,
llegan a un planeta y toman el control de
todo.

Por eso debemos detenerlos, ¡cuanto antes
mejor! Deben inyectar un código especial,
que nosotros, los zetarianos, les
proporcionaremos; esto en el servidor
central de los cerdos para que su imperio
finalmente caiga... Es eso o ¡terminarán
hablando PigLatin para toda la eternidad!

Teniendo ayuda de los extraterrestres el panorama pintaba un poco mejor que los días anteriores.



Sin embargo tu zozobra no iba a durar mucho...

Al día siguiente te escabulles al salón de informática, (es increíble lo lejos que se sienten los cerdos, estando en realidad tan cerca de la base escondida del salón 1 - piensas para tus adentros - Al menos tenemos un mecanismo bastante sofisticado con contraseña que no deja entrar intrusos, aunque a veces los compatriotas sean los que se quedan fuera - esbozas una sonrisa genuina, recordando como a tu amigo casi lo rapta un cerdo mutante la semana pasada por haber olvidado por enésima vez la contraseña de la puerta).

En fin, respiras profundo y te infiltras en el salón de informática, estás a punto de inyectar el código salvador, cuando...

SYNTAX ERROR - Line 1, Colum 1.

Claro, como pudimos ser tan ingenuos - piensas, mientras te chocas la frente con una palma - El código intermedio no es reconocido, el código debe ser en ensamblador.

Sales apresuradamente de vuelta a la guarida, la nueva misión es clara: utilizar el código intermedio para completar la traducción al lenguaje ensamblador.

Descripción de la aplicación

El proyecto 2 es la continuación del proyecto 1, se deberán utilizar las tripletas/cuartetas anteriormente generadas para traducir los lenguajes de alto nivel a lenguaje ensamblador funcional.



Descripción de los lenguajes

Y?

El lenguaje Y? se mantiene igual que en el proyecto 1.

Zetariano

Al lenguaje zetariano se le tendrán que agregar los siguientes elementos:

Encapsulamiento

Encapsulamiento para métodos y atributos de los objetos

- **public:** Accesible desde cualquier clase y paquete, sin restricciones.
- **private:** Accesible únicamente dentro de la misma clase donde fue declarado.
- **protected:** Accesible dentro del mismo paquete y por subclases (clases hijas), incluso si están en otros paquetes.
- **default (sin modificador):** Accesible únicamente dentro de las clases que pertenecen al mismo paquete (también llamado package-private).

Herencia

Las clases pueden heredar de otras, se asume que todas las clases



```
// Superclase (Clase padre) en el archivo correspondiente
public class Animal {
    String nombre;

    public void comer() {
        println(nombre + " está comiendo.");
    }
}

// Subclase (Clase hija) que hereda de Animal
public class Perro extends Animal {
    public void ladrar() {
        println(nombre + " está ladrando: ¡Guau, guau!");
    }
}
```

Polimorfismo

Los métodos de las clases pueden ser sobrescritas por otras. Para el proyecto se toma la anotación `@Override` como obligatoria para definir polimorfismo.

```
class Impresora {
    public void imprimir() {
        println("Imprimiendo documento genérico...");
    }
}

class ImpresoraPDF extends Impresora {
    @Override
    public void imprimir() {
```



```
        println("Imprimiendo en formato .PDF optimizado...");
    }
}

class ImpresoraExcel extends Impresora {
    @Override
    public void imprimir() {
        println("Imprimiendo hojas de cálculo en formato tabular...");
    }
}
```

Uso de la palabra reservada this

Al incluir herencia y polimorfismo

Pig Latin

Al lenguaje PigLatin se le tendrá que modificar lo siguiente:

Uso de objetos

```
esto miObjeto : novus Persona(12, "Profesor") : Persona ;

// la clase se toma en su forma Persona, porque es el tipo explícito
esto otroObjeto : novus Profesor(12, miObjeto, novus Persona()) : Persona ;

// Ahora las clases pueden ser nulas
esto otroObjeto : null : Persona ;
```



Características obligatorias de la práctica

- ☐ Análisis léxico y sintáctico del lenguaje, con el manejo adecuado de errores léxicos, sintácticos y semánticos.
- ☐ Interfaz amigable para el usuario
- ☐ Coloreado del código (los colores quedan a libertad del desarrollador) para los tres lenguajes y en tiempo real. No se permiten librerías para el coloreado.
- ☐ Gestión del árbol de trabajo (abrir, guardar y descargar, tanto carpetas como archivos)
- ☐ Documentación técnica
 - ☐ Tecnologías utilizadas
 - ☐ Diagrama de clases
 - ☐ Especificaciones de palabras reservadas, símbolos y gramática
 - ☐ Tabla de compatibilidad de tipos (a qué tipo se convierte si se suma un string con un int por ejemplo) para los tres lenguajes.
- ☐ Manual de usuario
- ☐ **Traducción final a RISC-V Assembler, debe ser funcional para que se tome como válido.**



Consideraciones importantes

- ☐ Realizar cualquier análisis sintáctico y semántico con ANTLR4
- ☐ Lenguaje de programación a utilizar: JAVA
- ☐ Si se va a utilizar código de internet entenderlo para que sea tomado como válido
- ☐ Copias obtendrán nota de 0

Entregables

- ☐ Código en Github
- ☐ Manuales técnico y de usuarios en el repositorio

Fecha de entrega xx de xxxx mediante la plataforma classroom.